TinkerWithMe

tinkerwithanne@gmail.com
Nairobi, Kenya

Smart Home Builder · Full Curriculum

3-session series (90 min each) · 10–14 yrs · Intermediate → Advanced

Full facilitator curriculum **PREMIUM**

Overview

Kids build a working smart home model with three integrated systems, wired into one house.

DETAIL	
Track	Arduino Robotics & Electronics
Format	3-session series (90 min each)
Ages	10–14 yrs
Level	Intermediate → Advanced
Price	from KES 8,000
Standalone	No
Prerequisites	Traffic light simulation OR Button controlled LED (free)

Course Overview

Smart Home Builder is a three-session Arduino intensive in which students design, wire, code and physically install three real automation systems into a single cardboard house model. Rather than building three disconnected demos, every session adds a new subsystem onto the *same* Arduino Uno and the *same* house shell, so by the end of Session 3 all three systems run together from one integrated sketch — exactly like a real smart home controller.

The arc moves from sensing-and-switching basics (light + motion controlling a relay and an LED) through data-driven automation (temperature triggering a fan, live readout on an LCD) to a full security system (ultrasonic distance sensing, an audible alarm, and a servo-actuated door lock). Each session revisits and extends the previous session's wiring and code, reinforcing debugging skills, pin planning,

and reading component datasheets, while building genuine system-integration confidence.

The take-home is a cardboard smart house model, built and decorated by the student across the three sessions, with all circuits mounted inside the walls/roof and one Arduino Uno running the final combined sketch (lighting + climate + security). Students leave with a working model, printed/soft copy of all their code, and the skills to keep extending it at home.

Learning Outcomes

- Wire and calibrate analogue sensors (LDR photoresistor) using a voltage-divider circuit
- Use digital input sensors (PIR, ultrasonic HC-SR04) and interpret their output in code
- Drive higher-power loads safely through a relay module, understanding active-LOW switching logic
- Read a digital temperature/humidity sensor (DHT11) and display live data on an I2C LCD
- Control a servo motor for a physical actuation task (a lockable door)
- Combine multiple independent subsystems into one non-blocking Arduino sketch using `millis()` timing instead of `delay()` where needed
- Plan, wire and physically install electronics into a fabricated model, including cable management and mounting

Materials Master List

ITEM	SPEC	QTY PER GROUP (2 STUDENTS)	NOTES
Arduino Uno R3 (or compatible)	ATmega328P, USB-B	1	Reused across all 3 sessions
USB cable	USB-A to USB-B	1	For programming/power
Breadboard	830-point, full size	1	Nerokas/Pixel Electronics stock this
Jumper wires	Male-male, assorted lengths, pack of 40	1 pack	

			Extra length needed once components move into the house walls
Photoresistor (LDR)	5mm GL5528 type	1	Session 1
Resistor	10kΩ, 1/4W	2	1 for LDR divider, spare
Resistor	220Ω, 1/4W	4	LED current-limiting
PIR motion sensor	HC-SR501	1	Session 1
LED	5mm, red (hallway) + yellow (room indicator)	2	Session 1
2-channel relay module	5V, active-LOW trigger	1	Ch1 = room light, Ch2 = fan
Small 5V DC fan	30–40mm brushless "muffin" fan	1	Session 2, switched via relay Ch2
DHT11 sensor module	3-pin breakout board	1	Session 2
16x2 LCD with I2C backpack	PCF8574 I2C interface, address 0x27 or 0x3F	1	Session 2
HC-SR04 ultrasonic sensor	4-pin (VCC, Trig, Echo, GND)	1	Session 3
Active buzzer	5V, 2-pin	1	Session 3
Micro servo	SG90, 180°	1	Session 3, door lock
Cardboard sheets	A2-size corrugated, 3–4 sheets	1 set	House shell — cut over the 3 sessions
Craft knife + cutting mat	Standard	1 shared per 2 groups	Facilitator supervises all cutting
Hot glue gun + glue sticks	Mini glue gun	1 shared per 2 groups	Facilitator supervises
Craft supplies	Paint, markers, popsicle sticks, tape	1 set	Decoration and door/hallway build
Power bank or 9V battery + barrel adapter	5V/9V, for standalone running	1 (optional)	Lets model run without laptop for showcase
Small screwdriver + double-sided tape	Precision flathead	1 set	Mounting sensors to cardboard
	IDE 2.x	1 per group	

Laptop with Arduino IDE installed	Libraries below installed in advance

Libraries to pre-install on all laptops before Session 2: "DHT sensor library" by Adafruit, "Adafruit Unified Sensor", and an I2C LCD library such as "LiquidCrystal I2C" by Frank de Brabander. "Servo" (for Session 3) ships built into the Arduino IDE.

Session Plans

Session 1: Smart Lighting

Objectives

- Explain how a voltage-divider circuit lets an LDR be read as an analogue signal
- Wire and program an active-LOW relay to switch a "room light" based on ambient light level
- Wire a PIR sensor to trigger a hallway LED with a timed hold, and mount both into the cardboard house

Timing (90 minutes)

- 0–10 min: Welcome, safety briefing (glue gun/craft knife rules), intro to smart homes and what we're building over 3 weeks
- 10–20 min: Concept talk — voltage dividers, analogRead(), what a relay does and why active-LOW modules exist
- 20–35 min: Guided breadboard wiring of LDR divider + relay + room LED, facilitator demos on a projector board first
- 35–50 min: Upload starter code, calibrate darkThreshold live using Serial Monitor readings under torch/room light
- 50–65 min: Wire and test PIR + hallway LED, tune HALLWAY_HOLD timing
- 65–80 min: Cut/assemble the base of the cardboard house (walls + roof outline), mark mounting spots for LDR (near a "window"), PIR (in the "hallway"), and plan cable routing to the Arduino which sits outside/under the house floor
- 80–90 min: Mount components with double-sided tape, quick group test, clean-up

Step-by-step

1. Open with a question: "How does a streetlight know when to turn on?" Lead into LDR resistance changing with light.

2. Draw the voltage divider on the whiteboard: 5V → LDR → junction (to A0) → 10kΩ resistor → GND. Explain that more light = lower LDR resistance = higher voltage at the junction (this specific LDR/resistor position gives HIGHER analog value in brighter light — confirm orientation with your kit and let students verify with Serial Monitor rather than memorising a rule).
3. Have students build the divider on the breadboard, connect to A0.
4. Introduce the relay module: explain the 3-pin logic side (VCC, GND, IN) and the 3-terminal switch side (COM, NO, NC). For this course we treat it as active-LOW: IN=LOW turns the relay ON.
5. Wire relay IN to pin 7, relay VCC/GND to Arduino 5V/GND. Wire the "room light" LED (through 220Ω resistor) across the relay's NO/COM terminals with a separate small battery loop OR simply drive the LED indicator directly if not using mains-style switching — for safety in this course the relay switches a low-voltage LED circuit only, never mains power.
6. Upload the Session 1 sketch. Open Serial Monitor, note the raw light values in the room vs under a jacket/torch, and adjust `darkThreshold` together as a class.
7. Wire the PIR sensor: VCC to 5V, GND to GND, OUT to pin 2. Turn the two onboard potentiometers (sensitivity/hold time) fully counter-clockwise for a fast, short-range demo response.
8. Wire the hallway LED (through 220Ω resistor) to pin 4 and GND.
9. Test: wave a hand near the PIR, confirm hallway LED lights for ~5 seconds after motion stops.
10. Move to the craft table: mark where the "window" (LDR) and "hallway" (PIR + LED) will sit on the cardboard shell, cut small holes for the sensor faces, and hot-glue mounts. Route wires through a slit in the floor down to the Arduino.

Build detail

Materials: Arduino Uno, breadboard, LDR, 10kΩ resistor, 2× 220Ω resistor, 2-channel relay module, 1 yellow LED (room), 1 red LED (hallway), PIR sensor, jumper wires, cardboard shell pieces, glue gun, craft knife.

Wiring/assembly (component → pin):

- LDR leg 1 → 5V; LDR leg 2 → A0 and → one leg of 10kΩ resistor; other leg of 10kΩ resistor → GND

- Relay module VCC → 5V; Relay module GND → GND; Relay module IN1 → digital pin 7
- Room LED (via 220Ω resistor) wired across relay NO/COM output terminals, with LED's own small supply loop from 5V/GND through the relay switch path
- PIR VCC → 5V; PIR GND → GND; PIR OUT → digital pin 2
- Hallway LED long leg → 220Ω resistor → digital pin 4; short leg → GND

```
/*
  TinkerWithMe - Smart Home Builder
  SESSION 1: Smart Lighting System
  - Photoresistor (LDR) + Relay -> auto room light at dusk
  - PIR motion sensor -> hallway LED
*/

#define LDR_PIN       A0     // LDR voltage divider output
#define ROOM_RELAY    7      // Relay module IN1 - controls room light
#define PIR_PIN       2      // PIR sensor OUT
#define HALLWAY_LED   4      // Hallway LED (through 220 ohm resistor)

int darkThreshold = 500;     // Adjust after testing with your LDR (0-1023). Lower = darker.
unsigned long motionTime = 0;
const unsigned long HALLWAY_HOLD = 5000; // keep hallway light on 5s after motion

void setup() {
  pinMode(ROOM_RELAY, OUTPUT);
  pinMode(PIR_PIN, INPUT);
  pinMode(HALLWAY_LED, OUTPUT);

  // This relay module is ACTIVE LOW: LOW = relay ON, HIGH = relay OFF
  digitalWrite(ROOM_RELAY, HIGH);  // start with room light OFF
  digitalWrite(HALLWAY_LED, LOW);

  Serial.begin(9600);
  Serial.println("Smart Lighting System ready");
}

void loop() {
  int lightLevel = analogRead(LDR_PIN);
  Serial.print("Light level: ");
  Serial.println(lightLevel);

  // --- ROOM LIGHT LOGIC ---
  if (lightLevel < darkThreshold) {
    digitalWrite(ROOM_RELAY, LOW);   // dark -> turn room light ON
  } else {
    digitalWrite(ROOM_RELAY, HIGH);  // bright -> turn room light OFF
  }

  // --- HALLWAY MOTION LOGIC ---
  int motionDetected = digitalRead(PIR_PIN);
  if (motionDetected == HIGH) {
    digitalWrite(HALLWAY_LED, HIGH);
    motionTime = millis();           // reset timer every time motion is seen
  }
  if (millis() - motionTime > HALLWAY_HOLD) {
```

```
    digitalWrite(HALLWAY_LED, LOW);
  }

  delay(200); // small delay for stable readings
}
```

Checks for understanding

- Why does the LDR need a second resistor in the circuit — what would happen without it?
- What does "active-LOW" mean for our relay, and how did that change our code?
- If the hallway light turns off too quickly, which variable would you change and in which direction?

MISTAKE	FIX
LDR reads the same value regardless of light	Check the divider is actually connected to A0, not floating; confirm 10kΘ resistor leg is in GND rail, not a dead breadboard row
Relay clicks but LED never lights	Confirm LED polarity and that the LED's own power loop is wired through the relay's switched terminals, not just the logic side
PIR triggers constantly / never resets	Turn onboard time-delay potentiometer fully counter-clockwise; add a warm-up wait of 30–60s after power-on before testing (PIR needs to stabilise)

Session 2: Climate Control

Objectives

- Read temperature and humidity from a DHT11 sensor and understand digital sensor timing limits
- Display live sensor data on an I2C LCD
- Extend the Session 1 circuit with a second relay channel to auto-control a fan by temperature

Timing (90 minutes)

- 0–10 min: Recap Session 1 circuit, confirm it still runs; introduce today's goal (climate control)
- 10–20 min: Concept talk — digital vs analogue sensors, why DHT11 needs a delay between reads, I2C bus basics (2 wires, address)
- 20–35 min: Wire DHT11, upload a temperature-only test sketch, read values in Serial Monitor

- 35–50 min: Wire I2C LCD (SDA/SCL), run an "I2C scanner" if the LCD shows nothing, get text on screen
- 50–65 min: Wire fan to relay channel 2, merge into combined climate + lighting sketch, calibrate tempThreshold using body heat/hand near sensor
- 65–80 min: Cut a "vent" opening in the cardboard house, mount fan behind it, mount LCD on an interior "wall" visible through a cut window, route DHT11 wire to sit near the vent
- 80–90 min: Full test of lighting + climate together, troubleshoot as groups, clean-up

Step-by-step

1. Recap: reconnect yesterday's breadboard, briefly re-run Session 1 sketch to confirm nothing broke over the week.
2. Explain the DHT11 is a digital sensor sending a timed data packet on one wire — that's why we can't read it faster than ~once per second, unlike the LDR.
3. Wire DHT11 (VCC, GND, DATA→pin 3). Load a minimal test sketch using the DHT library, confirm Serial Monitor shows sensible numbers (~20–30°C, 30–70% humidity indoors).
4. Introduce I2C: only 2 signal wires (SDA=A4, SCL=A5) can control many devices because each has an address. Wire the LCD, run `lcd.init(); lcd.backlight();` and print "Hello". If blank, try address 0x3F instead of 0x27.
5. Wire fan + relay channel 2 (pin 6) exactly as the room light relay was wired in Session 1, but to a small fan motor instead of an LED.
6. Merge all four subsystems (LDR/relay1/PIR/LED from Session 1 + DHT11/LCD/relay2/fan) into one sketch — walk through the Session 2 code together, pointing out how we use `millis()` instead of `delay()` so the DHT11's slow reads don't freeze the lighting logic.
7. Calibrate: cup hands around the DHT11 to raise its reading and watch the fan relay click on once it passes tempThreshold.
8. Move to craft table: cut a vent hole, position fan so airflow is visible/felt, cut a window for the LCD to be read from outside the house, secure DHT11 near the vent opening (not touching the fan motor, to avoid heat interference).

Build detail

Materials: everything from Session 1 (still wired in), plus DHT11 module, 16x2 I2C LCD, small 5V fan, extra jumper wires.

Wiring/assembly (component → pin):

- DHT11 VCC → 5V; DHT11 GND → GND; DHT11 DATA → digital pin 3
- LCD I2C module VCC → 5V; GND → GND; SDA → A4; SCL → A5
- Relay module Ch2 IN2 → digital pin 6; fan wired across Ch2's switched terminals with its own supply loop from 5V/GND

```
#include
#include
#include

// ---- LIGHTING (from Session 1) ----
#define LDR_PIN       A0
#define ROOM_RELAY    7
#define PIR_PIN       2
#define HALLWAY_LED   4

// ---- CLIMATE (new) ----
#define DHTPIN        3
#define DHTTYPE       DHT11
#define FAN_RELAY     6

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2); // try 0x3F if screen stays blank

int darkThreshold = 500;
unsigned long motionTime = 0;
const unsigned long HALLWAY_HOLD = 5000;

float tempThreshold = 28.0;             // degrees C - fan turns on above this
unsigned long lastTempRead = 0;
const unsigned long TEMP_INTERVAL = 2000; // DHT11 needs >=1s between reads

void setup() {
  pinMode(ROOM_RELAY, OUTPUT);
  pinMode(FAN_RELAY, OUTPUT);
  pinMode(PIR_PIN, INPUT);
  pinMode(HALLWAY_LED, OUTPUT);

  digitalWrite(ROOM_RELAY, HIGH); // relays are active-LOW: HIGH = OFF
  digitalWrite(FAN_RELAY, HIGH);
  digitalWrite(HALLWAY_LED, LOW);

  dht.begin();
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Smart Home v2");
  delay(1500);
  lcd.clear();

  Serial.begin(9600);
```

```
}

void loop() {
  // ---------- LIGHTING ----------
  int lightLevel = analogRead(LDR_PIN);
  if (lightLevel < darkThreshold) digitalWrite(ROOM_RELAY, LOW);
  else digitalWrite(ROOM_RELAY, HIGH);

  int motionDetected = digitalRead(PIR_PIN);
  if (motionDetected == HIGH) {
    digitalWrite(HALLWAY_LED, HIGH);
    motionTime = millis();
  }
  if (millis() - motionTime > HALLWAY_HOLD) {
    digitalWrite(HALLWAY_LED, LOW);
  }

  // ---------- CLIMATE ----------
  if (millis() - lastTempRead > TEMP_INTERVAL) {
    lastTempRead = millis();
    float temp = dht.readTemperature();
    float hum = dht.readHumidity();

    if (isnan(temp) || isnan(hum)) {
      lcd.setCursor(0, 0);
      lcd.print("Sensor error!  ");
    } else {
      lcd.setCursor(0, 0);
      lcd.print("Temp: ");
      lcd.print(temp, 1);
      lcd.print((char)223); // degree symbol
      lcd.print("C   ");

      lcd.setCursor(0, 1);
      lcd.print("Hum: ");
      lcd.print(hum, 0);
      lcd.print("%    ");

      if (temp > tempThreshold) digitalWrite(FAN_RELAY, LOW);  // ON
      else digitalWrite(FAN_RELAY, HIGH);                      // OFF

      Serial.print("Temp: "); Serial.print(temp);
      Serial.print(" Hum: "); Serial.println(hum);
    }
  }

  delay(100);
}
```

Checks for understanding

- Why can't we read the DHT11 every loop cycle the way we read the LDR?
- What do SDA and SCL do, and why can several devices share just those two wires?

- If the fan never turns on, what are two different possible causes you could check?

MISTAKE	FIX
DHT11 returns "nan" constantly	Check DATA wire is on pin 3 and matches `DHTPIN`; some DHT11 breakouts need a 10kΘ pull-up on DATA if using the bare sensor instead of a module
LCD backlight on but no text	Adjust the contrast potentiometer on the back of the I2C backpack; try alternate address 0x3F
Fan runs constantly regardless of temperature	Check relay wiring polarity (active-LOW logic) and confirm tempThreshold is below current room temperature during testing

Session 3: Security System

Objectives

- Use an ultrasonic sensor to measure distance and detect someone approaching a door
- Trigger a buzzer alarm and control a servo-based door lock from code
- Integrate lighting, climate and security into one final sketch and complete the house model

Timing (90 minutes)

- 0–10 min: Recap Sessions 1–2, confirm circuit still works; introduce the security brief ("keep the house safe")
- 10–20 min: Concept talk — how ultrasonic distance sensing works (speed of sound, pulse timing), servo angles vs continuous motors
- 20–35 min: Wire HC-SR04, run a distance-only test sketch, read live cm values in Serial Monitor
- 35–45 min: Wire buzzer and servo, test each independently (buzzer beep test, servo sweep test)
- 45–65 min: Merge into the final combined sketch (lighting + climate + security); test lock/alarm behaviour using Serial Monitor commands
- 65–80 min: Final house assembly — mount ultrasonic sensor at the "front door" facing outward, mount buzzer inside near the door, attach servo with a small cardboard/lolly-stick latch arm onto the door itself
- 80–90 min: Full system test of the complete smart house, tidy wiring, prep for showcase

Step-by-step

1. Recap circuit, reconnect from Session 2, confirm lighting and climate systems still behave correctly.
2. Explain HC-SR04: it sends an ultrasonic "chirp" from Trig, and Echo goes HIGH for exactly as long as it took the sound to bounce back — we convert that time into distance using the speed of sound.
3. Wire Trig→pin 9, Echo→pin 10. Load a distance test sketch, hold a hand at different distances and watch Serial Monitor numbers change.
4. Wire buzzer to pin 11, test with a simple `digitalWrite(BUZZER_PIN, HIGH)` beep.
5. Wire servo to pin 12 (signal), 5V, GND. Test with `doorServo.write(0)` and `doorServo.write(90)` to find lock/unlock angles that suit the physical latch students will build.
6. Walk through the final combined sketch as a class, focusing on the new security block: how the alarm auto-stops after ALARM_DURATION, and how typing 'U' or 'L' into Serial Monitor simulates a keypad/app unlocking the door (name this as an extension opportunity for home).
7. Have each group agree on DOOR_ALARM_DISTANCE for their sensor's mounting position and tune it live.
8. Craft finishing: cut a small slot for the ultrasonic sensor face at "door height" on the front wall, mount the buzzer inside the front wall cavity, and build a simple cardboard/lolly-stick door with a latch arm glued to the servo horn so rotating the servo visibly locks/unlocks the door.
9. Run the full showcase-ready system: darken the room to trigger lighting, wave near the PIR, warm the DHT11, and approach the front door to trigger the alarm — all in the same physical house.

Build detail

Materials: everything from Sessions 1–2 (still wired in), plus HC-SR04 ultrasonic sensor, active buzzer, SG90 servo, lolly sticks/cardboard strip for the door latch, extra jumper wires.

Wiring/assembly (component → pin):

- HC-SR04 VCC → 5V; GND → GND; Trig → digital pin 9; Echo → digital pin 10
- Buzzer positive leg → digital pin 11; negative leg → GND
- Servo signal wire → digital pin 12; power wire → 5V; ground wire → GND

- Servo horn glued to a lolly-stick latch arm which slides across the door frame when the servo rotates between LOCK_ANGLE and UNLOCK_ANGLE

```
#include
#include
#include
#include

// ---- LIGHTING ----
#define LDR_PIN       A0
#define ROOM_RELAY    7
#define PIR_PIN       2
#define HALLWAY_LED   4

// ---- CLIMATE ----
#define DHTPIN        3
#define DHTTYPE       DHT11
#define FAN_RELAY     6

// ---- SECURITY ----
#define TRIG_PIN      9
#define ECHO_PIN      10
#define BUZZER_PIN    11
#define SERVO_PIN     12

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo doorServo;

int darkThreshold = 500;
unsigned long motionTime = 0;
const unsigned long HALLWAY_HOLD = 5000;

float tempThreshold = 28.0;
unsigned long lastTempRead = 0;
const unsigned long TEMP_INTERVAL = 2000;

const int DOOR_ALARM_DISTANCE = 15;  // cm - closer than this while locked = alarm
const int LOCK_ANGLE = 0;            // servo angle: door LOCKED
const int UNLOCK_ANGLE = 90;         // servo angle: door UNLOCKED
bool doorLocked = true;
bool alarmActive = false;
unsigned long alarmStart = 0;
const unsigned long ALARM_DURATION = 4000;

void setup() {
  pinMode(ROOM_RELAY, OUTPUT);
  pinMode(FAN_RELAY, OUTPUT);
  pinMode(PIR_PIN, INPUT);
  pinMode(HALLWAY_LED, OUTPUT);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(ROOM_RELAY, HIGH);
  digitalWrite(FAN_RELAY, HIGH);
  digitalWrite(HALLWAY_LED, LOW);
  digitalWrite(BUZZER_PIN, LOW);
```

```
  dht.begin();
  lcd.init();
  lcd.backlight();

  doorServo.attach(SERVO_PIN);
  doorServo.write(LOCK_ANGLE);
  doorLocked = true;

  Serial.begin(9600);
  lcd.setCursor(0, 0);
  lcd.print("Smart Home ON");
  delay(1500);
  lcd.clear();
}

long readDistanceCM() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30ms timeout (~5m max)
  if (duration == 0) return 999; // nothing detected in range
  long distance = duration * 0.034 / 2; // speed of sound = 0.034 cm/us
  return distance;
}

void loop() {
  // ---------- LIGHTING ----------
  int lightLevel = analogRead(LDR_PIN);
  if (lightLevel < darkThreshold) digitalWrite(ROOM_RELAY, LOW);
  else digitalWrite(ROOM_RELAY, HIGH);

  int motionDetected = digitalRead(PIR_PIN);
  if (motionDetected == HIGH) {
    digitalWrite(HALLWAY_LED, HIGH);
    motionTime = millis();
  }
  if (millis() - motionTime > HALLWAY_HOLD) {
    digitalWrite(HALLWAY_LED, LOW);
  }

  // ---------- CLIMATE ----------
  if (millis() - lastTempRead > TEMP_INTERVAL) {
    lastTempRead = millis();
    float temp = dht.readTemperature();
    float hum = dht.readHumidity();
    if (!isnan(temp) && !isnan(hum)) {
      lcd.setCursor(0, 0);
      lcd.print("T:"); lcd.print(temp, 1); lcd.print((char)223); lcd.print("C ");
      lcd.setCursor(8, 0);
      lcd.print("H:"); lcd.print(hum, 0); lcd.print("% ");

      if (temp > tempThreshold) digitalWrite(FAN_RELAY, LOW);
      else digitalWrite(FAN_RELAY, HIGH);
    }
  }
```

```
  // ---------- SECURITY ----------
  long distance = readDistanceCM();

  lcd.setCursor(0, 1);
  lcd.print("Door:"); lcd.print(distance); lcd.print("cm  ");

  if (distance < DOOR_ALARM_DISTANCE && doorLocked) {
    if (!alarmActive) {
      alarmActive = true;
      alarmStart = millis();
    }
    digitalWrite(BUZZER_PIN, HIGH);
  }

  if (alarmActive && millis() - alarmStart > ALARM_DURATION) {
    alarmActive = false;
    digitalWrite(BUZZER_PIN, LOW);
  }

  // Type U to unlock or L to lock in the Serial Monitor - simulates a keypad/app
  if (Serial.available()) {
    char cmd = Serial.read();
    if (cmd == 'U' || cmd == 'u') {
      doorServo.write(UNLOCK_ANGLE);
      doorLocked = false;
      digitalWrite(BUZZER_PIN, LOW);
      alarmActive = false;
      Serial.println("Door UNLOCKED");
    }
    if (cmd == 'L' || cmd == 'l') {
      doorServo.write(LOCK_ANGLE);
      doorLocked = true;
      Serial.println("Door LOCKED");
    }
  }

  delay(150);
}
```

Checks for understanding

- Why do we multiply the pulse duration by 0.034 and then divide by 2 to get distance in cm?
- What happens in our code if the door is unlocked and someone walks up close — why doesn't the alarm sound?
- Name one real risk of using `delay()` everywhere instead of `millis()` in a system with this many sensors.

MISTAKE	FIX
Distance reading is always 999 or wildly inconsistent	Check Trig/Echo aren't swapped; make sure nothing is touching/blocking the sensor face; confirm common GND between sensor and Arduino

Servo jitters or resets randomly	Servo is drawing too much current from the Uno's 5V pin — power it from a separate 5V source (power bank/battery pack) sharing GND with the Arduino, especially once other components are also active
Alarm never stops sounding	Check ALARM_DURATION logic — confirm alarmStart is being set only once per trigger, not reset every loop; confirm distance rises back above threshold once the "visitor" steps back

Assessment & Showcase

Assess students continuously across the three sessions rather than with a single end test, using a simple rubric per subsystem: **Wired correctly**, **Code runs without errors**, **Behaviour matches the brief** (e.g. light turns on only when dark, alarm only sounds when locked). Facilitators should tick each subsystem off in a tracking sheet at the end of each session, and note which students calibrated thresholds independently versus needed guidance — this is the strongest signal of real understanding versus copied code.

At the end of Session 3, run a "Smart Home Showcase": each group presents their house to the rest of the class and to any visiting parents. A completed model is "done" when, live and unprompted, it can: (1) turn on the room light when the room is darkened, (2) light the hallway briefly on motion, (3) show live temperature/humidity on the LCD and switch the fan on when warmed, and (4) sound an alarm when something approaches the locked front door, with the facilitator or student able to "unlock" it via Serial Monitor to stop the alarm and open the door. Groups narrate what each sensor does and one thing they debugged themselves during the course — this speaking component is part of the assessment, not just the working circuit.

Extension & Home Challenges

- **Easy:** Add a second hallway LED further down the "corridor" and wire it to the same PIR output, so motion lights two spots at once. Adjust HALLWAY_HOLD so the lights stay on longer at night.
- **Medium:** Replace the Serial Monitor lock/unlock commands with a physical push-button or a 4-pin keypad module wired to a spare digital pin, so the door can be unlocked without a laptop connected — this teaches simple keypad or button debounce logic to add into the existing sketch.

- **Hard:** Add a second ultrasonic sensor at the back of the house and combine both distance readings so the alarm reports (via Serial Monitor or a second LED) which side of the house the intruder approached from, and extend the LCD to alternate between showing temperature and a "last alarm" timestamp using `millis()`.